

PROJECT VISION DOCUMENT — V3.0

Rock Paper Scissors Universe

From a single C++ file to a cross-platform tournament engine with local play, Discord integration, Rust networking, Zig media tools, and MemeForge AI reactions

Version: 3.0

Date: Sat June 13 2026

1. Project Overview

Rock Paper Scissors Universe is a multi-phase project that starts as a clean C++ terminal game and grows into a cross-platform tournament system.

The project begins with a simple Rock Paper Scissors game written in C++, but the long-term vision is much bigger:

- local multiplayer,
- online multiplayer,
- tournaments,
- Discord integration,
- Telegram integration,
- player rankings,
- funny AI reactions,
- meme generation,
- GIFs,
- short videos,
- audio reactions,
- social sharing,
- merch,
- and reusable game plugins.

The project is not only a game. It is a small competitive social universe built around one of the simplest games in human history.

The main idea is:

A tiny decision becomes a legendary battle.

Players can compete locally in the same room without internet, or online through Discord and later other platforms. The same core game engine powers every mode.

The project also includes reusable engines:

- a tournament bracket engine,
- a scoring and ranking system,
- and MemeForge, an AI-powered reaction engine that creates personalized memes, GIFs, videos, audio reactions, and shareable game moments.

1.1 Starting Point and Long-Term Goal

Area	Description
Started from	Single-file C++ Rock Paper Scissors game
First goal	Clean multi-file C++ terminal game
Long-term goal	Cross-platform tournament app and reusable plugin system
Core engine	C++
Networking/backend	Rust
Media/audio/GIF tools	Zig
First online platform	Discord bot
Future platforms	Telegram, Instagram, web, mobile
Local connectivity	WiFi, hotspot, Bluetooth
Online connectivity	Discord, Telegram, invite links, PIN codes
AI reaction engine	MemeForge

Offline meme mode Pre-installed meme pack

Online meme mode AI-generated captions, images, GIFs, videos, and audio

1.2 Core Philosophy

The project should be built step by step.

The first version should not try to do everything. The correct path is:

First make the game clean.

Then make the engine reusable.

Then add tournaments.

Then add networking.

Then add Discord.

Then add MemeForge.

Then add social sharing and merch.

The project must stay organized.

The C++ game engine should not directly know about Discord, Telegram, AI, database systems, or memes. It should only understand the game itself:

- players,
- choices,
- rounds,
- winners,
- losers,
- draws,
- sessions,
- tournaments,
- scores.

Other systems should connect to it through clean interfaces.

2. Game Modes

2.1 Tournament Mode

Tournament Mode is the main game mode.

Any number of players can join a session. The system randomly pairs players into 1v1 matches each round.

The loser is eliminated. The winner advances.

This continues until one final winner remains.

Tournament Mode should feel like an arena. Every round should feel dramatic, funny, and competitive.

2.2 Odd-Player Handling

If the number of players is odd, one player will not have a human opponent during that round.

The system handles this by pairing the leftover player against the computer.

Rules:

- each round players are shuffled randomly,
- players are paired in order,
- if one player is left without an opponent, they face the computer,
- if the human wins, they advance,
- if the computer wins, the computer advances as a slot,
- the final match can be human vs human or human vs computer.

This makes tournaments work with any number of players.

2.3 Quick Decision Mode

Quick Decision Mode is a lightweight mode.

It is not a full tournament.

It is used when people want to decide something quickly.

Examples:

- who pays for food,
- who chooses the movie,

- who goes first,
- who takes the last slice,
- who loses a bet,
- who has to do a small task.

Quick Decision Mode is simple:

- one round only,
- two or more players,
- instant result,
- no bracket,
- no elimination,
- no long setup.

The result should feel funny and final.

Example:

The gods have spoken.

Zain pays today.

May his bank account recover with honor.

3. Round Timing, Reveal Psychology, and Addiction Loop

The game should not reveal choices immediately after each player selects Rock, Paper, or Scissors.

The strongest experience is created through:

Private choice lock-in + short reveal countdown + simultaneous reveal.

This makes the game feel fair, dramatic, and addictive.

3.1 Recommended Round Flow

Each round works like this:

1. Players privately choose Rock, Paper, or Scissors.
2. Their choices are locked secretly.
3. The game waits until all required players have locked in.
4. A short reveal ritual begins:

3...

2...

1...

GO!

5. All choices are revealed at the same time.
 6. The winner, loser, draw, score update, and meme reaction are shown.
-

3.2 Why This Feels Good

This system combines two strong psychological effects.

The countdown creates:

- urgency,
- excitement,
- instinctive decisions,
- pressure,
- ritual.

The simultaneous reveal creates:

- suspense,
- fairness,
- surprise,
- drama,
- social reaction.

The countdown should not be used as the main technical synchronization system.

The countdown is mainly a user-experience ritual.

The real system should depend on the host, bot, or server controlling the state.

3.3 Why Not Pure Countdown?

A pure countdown system creates panic, but it is harder to build fairly.

If players are using Bluetooth, WiFi, Discord, or slow internet, their devices may not stay perfectly synchronized.

Even a small delay can make the game feel unfair.

So the game should not depend on millisecond-perfect timing.

3.4 Why Simultaneous Reveal Is Better

Simultaneous reveal is easier and more reliable.

Each player sends their choice privately to the host, bot, or server.

The result is hidden until everyone has submitted their choice.

This creates a feeling similar to poker:

Everyone acts secretly.

Then everyone reveals at the same time.

The moment before the reveal is one of the most important emotional moments in the game.

3.5 Optional Timer

A timer can still exist, but it should be forgiving.

Example:

Players have 10–15 seconds to lock in their choice.

If a player does not choose in time, the system can:

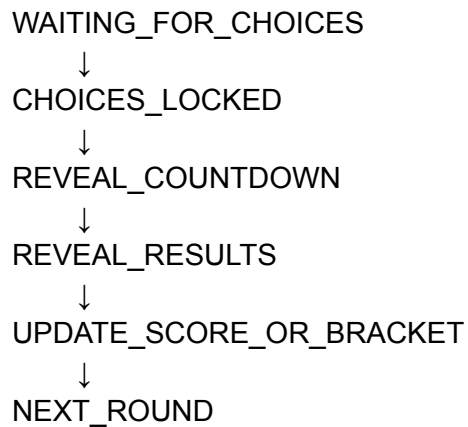
- auto-pick randomly,
- mark them as inactive,
- or make them lose the round.

For casual play, auto-random is better because it keeps the game moving.

For competitive tournaments, missing the timer can count as a loss.

3.6 Technical Round State Model

The recommended technical model is:



The important rule:

The game should synchronize the reveal, not the decision moment.

This makes the system easier to support across:

- terminal mode,
 - local WiFi mode,
 - Bluetooth mode,
 - Discord bot mode,
 - Telegram mode,
 - future mobile or web versions.
-

4. Connectivity — Local and Online

The game supports two main connectivity modes:

- local mode,

- online mode.

Both modes use the same core game engine.

The connectivity layer is an adapter.

The game logic never changes. Only the way players send choices and receive results changes.

4.1 Local Mode — No Internet Required

Local Mode is for players who are physically near each other.

Examples:

- same room,
- same house,
- same university,
- same gaming event,
- same restaurant,
- same 42 campus.

No internet connection is required.

One device acts as the host. Other players connect to it directly.

4.2 Local Connection Methods

Local Mode can support:

- WiFi on the same network,
- mobile hotspot,
- Bluetooth.

WiFi

The host starts a local server.

Other players connect through the local IP address, session code, PIN, or QR code.

Mobile Hotspot

One phone creates a hotspot.

Other players join the hotspot.

After that, the flow works like local WiFi.

Bluetooth

Bluetooth is useful for short-range play.

It is slower and more limited, but it works without a router and without internet.

It is best for small groups.

4.3 Local Join Methods

Players can join using:

- PIN code,
- QR code,
- direct local address,
- nearby discovery.

PIN Code

The host generates a short code.

Example:

482913

Players enter the code to join.

QR Code

The host displays a QR code.

Players scan it with their phones and join instantly.

Both PIN and QR should be available.

The host chooses which one to show.

4.4 Online Mode — Internet Required

Online Mode allows players to join from anywhere.

They do not need to be in the same location.

Online Mode can work through:

- Discord,
- Telegram,
- invite links,
- web app,
- mobile app,
- future social platforms.

4.5 Online Join Methods

Players can join using:

- invite link,
- PIN code,
- QR code,
- Discord slash command,
- Telegram command.

Example Discord command:

`/join 482913`

Example Telegram command:

`/join 482913`

The command is platform-specific, but the game engine underneath stays the same.

4.6 Connectivity Architecture

The core game engine should not care whether the game is local or online.

It should only receive a session object containing:

- player list,
- player choices,
- round state,
- tournament state,
- match result.

The connectivity layer fills that session object.

Example:

Player sends choice



Connectivity layer receives it



Session object is updated



Game engine calculates result



Connectivity layer sends result back

5. Platform Strategy

The game should be platform-agnostic.

Each social platform is an adapter.

The core game engine stays unchanged.

Adding a new platform should mean writing a new adapter, not rewriting the game.

5.1 Platform Priority

Priority	Platform	Reason
1	Discord	Strong gaming communities, bots, slash commands, embeds, GIF support

2	Telegram	Open bot API, easy joining, strong international user base
3	Web	Universal access, easy invite links
4	Mobile	Best long-term user experience
5	Instagram / Meta	Harder APIs, but strong sharing potential
6	Others	WhatsApp, Slack, university platforms, etc.

5.2 Platform Adapter Interface

Each platform adapter should implement the same type of behavior.

Example interface idea:

```
receive_choice(player_id) → GameChoice  
send_result(session, round_info) → void  
send_reaction(player_id, meme_file) → void  
get_player_profile(player_id) → PlayerProfile  
send_join_code(session_id, method) → void
```

The engine calls the interface.

The platform adapter decides how to display the result.

Discord may use embeds.

Telegram may use messages.

Terminal mode may use text output.

The game logic stays unchanged.

6. Multi-Language Technology Strategy

The project will not depend on only one programming language forever.

Each language should be used where it is strongest.

The goal is not to use many languages for no reason, but to build a serious system where each part is written with the best tool for that job.

6.1 C++ — Core Game Engine and Performance Logic

C++ remains the foundation of the first version of the project.

It is used for:

- the original Rock Paper Scissors game logic,
- player choice handling,
- winner calculation,
- tournament rules,
- score calculation,
- fast local terminal version,
- reusable game engine modules,
- performance-sensitive logic.

C++ is a good choice because the project started from a C++ game, and it allows strong control over memory, performance, structure, and low-level behavior.

The C++ engine should stay clean and platform-independent.

It should not directly depend on:

- Discord,
- Telegram,
- AI APIs,
- database code,
- networking code,
- meme generation code.

It should only know about:

- players,
- choices,
- rounds,
- sessions,
- scores,
- tournaments,
- results.

The rule is:

C++ owns the game brain.

6.2 Rust — Networking, Server Logic, and Safe Concurrency

Rust is a strong choice for the networking and backend side of the project.

Rust should be considered for:

- local multiplayer networking,
- online session hosting,
- server-side tournament management,
- player session state,
- matchmaking services,
- WebSocket communication,
- API services,
- safe concurrent player handling,
- secure backend logic,
- communication between clients and the game engine,
- database access layer.

Rust is especially useful because multiplayer games involve many players connecting at the same time.

This creates problems such as:

- race conditions,
- invalid states,
- dropped connections,
- synchronization bugs,
- unsafe shared state,
- connection timing issues.

Rust helps reduce these problems because it is designed around memory safety and safe concurrency.

The rule is:

Rust owns the network and session layer.

Rust can also manage the database access layer, but the database itself will probably be a real database system such as:

- PostgreSQL,
- SQLite,
- Redis.

Possible Rust flow:

Player connects



Rust server receives request



Rust validates session and player state



Rust sends game action to C++ core



C++ calculates result



Rust sends result back to players

6.3 Zig — Media, Audio, GIF, and Low-Level Creative Tools

Zig can be used for special performance-heavy tools around media generation.

Possible Zig use cases:

- audio generation,
- sound effect processing,
- funny loser sounds,
- victory sounds,
- small voice reaction tools,
- GIF frame processing,
- meme rendering tools,
- image/video pipeline helpers,
- lightweight command-line media tools,
- performance-focused asset processing.

Zig is interesting because it gives low-level control like C, but with a modern design.

It can be useful for small, fast tools that do one specific job very well.

For example, MemeForge may later generate:

- a short victory sound,
- a funny loser sound,
- a meme GIF,
- a short animated clip,
- a reaction video frame sequence.

Zig can be tested for these parts.

The rule is:

Zig owns the fast creative media tools.

6.4 Database and Storage

The project will need storage for:

- player accounts,
- stats,
- rankings,
- memes,
- tournaments,
- match history,
- saved reactions,
- social sharing,
- merch data.

Possible storage systems:

- SQLite for early local development,
- PostgreSQL for serious online backend,
- Redis for fast temporary session state,
- object storage for generated images, GIFs, and videos.

The database stores:

- player profiles,
- usernames,
- ranks,
- points,
- match history,
- rivalry history,
- tournament results,
- unlocked titles,

- favorite generated memes,
- saved GIFs or videos,
- social sharing metadata.

Rust can be used to build the service that communicates with the database safely.

6.5 Recommended Architecture by Language

C++:

Core game logic, round rules, tournament calculation, performance engine.

Rust:

Networking, backend server, sessions, matchmaking, database access, safe concurrency.

Zig:

Audio generation, GIF/video processing tools, fast meme/media rendering helpers.

Database:

PostgreSQL / SQLite / Redis depending on project stage.

AI APIs:

Meme text, image generation, video generation, personality reactions.

The project should begin simple with C++, then slowly add Rust and Zig only when the project actually needs them.

The correct order is:

Phase 1: C++ terminal game

Phase 2: C++ tournament engine

Phase 3: Rust local/online networking layer

Phase 4: Rust backend + database

Phase 5: MemeForge AI reaction system

Phase 6: Zig media tools for audio/GIF/video generation

This keeps the project realistic and avoids making it too complicated too early.

7. Game Story and World Identity

The game should not feel like a normal Rock Paper Scissors app.

It should feel like a dramatic ancient ritual used to solve ridiculous modern problems.

The story should be funny, exaggerated, and memorable.

The game should treat small decisions like world-ending historical events.

7.1 Main Game Lore

Long ago, before lawyers, voting systems, online polls, and group chats, humanity had one sacred method for solving impossible conflicts:

Rock.

Paper.

Scissors.

Kings used it before war.

Spartans used it before battle.

Philosophers used it when deciding who had to pay for olives.

Ancient restaurant groups used it when the bill arrived and everyone suddenly became poor.

The greatest warriors of history did not always fight with swords.

Sometimes they stood in a circle, stared into each other's souls, and whispered:

Best of three?

From the temples of Greece to the modern tables of restaurants, from student apartments to Discord servers, one truth survived:

When humans cannot decide, they must duel.

Rock Paper Scissors Universe turns every small decision into a legendary battle.

Who pays for dinner?

Who chooses the movie?

Who takes the last slice?

Who goes first?

Who loses the bet?

Who becomes the champion of the server?

Every tiny decision becomes a historical event.

7.2 Funny Exaggerated Game Intro

Possible intro text for the app:

In the ancient days, when kings feared bills, warriors feared group projects, and philosophers feared choosing where to eat, one sacred ritual ruled them all.

No sword.

No court.

No argument.

Only Rock.

Only Paper.

Only Scissors.

Now the ritual returns.

Choose wisely.

Your honor, your ranking, and possibly your dinner money depend on it.

7.3 Tournament Story Style

Tournament Mode should feel like a dramatic arena.

Example tournament intro:

The arena is ready.

Players have entered from every corner of the server.

Some came for glory.

Some came for revenge.
Some came because they did not want to pay for pizza.

Only one will survive.

Let the tournament begin.

Example elimination message:

Zain has fallen.

History will remember his courage.
The restaurant bill will remember his wallet.

Example final winner message:

Adam stands alone.

Champion of the arena.
Master of impossible decisions.
Enemy of unpaid restaurant bills.

7.4 Quick Decision Mode Story Style

Quick Decision Mode should be fast and funny.

Example:

A great conflict has appeared.

The people cannot agree.
The group chat is collapsing.
The waiter is waiting.

There is only one solution.

Rock Paper Scissors.

After the result:

The gods have spoken.

Zain pays today.

May his bank account recover with honor.

7.5 Brand Personality

The game personality should be:

- funny,
- dramatic,
- slightly chaotic,
- competitive,
- meme-friendly,
- social,
- easy to share,
- exaggerated on purpose.

The brand feeling is:

Tiny decisions. Legendary battles.

8. Community, Ranking, and Addiction System

The long-term goal is not only to build a Rock Paper Scissors game.

The goal is to build a recognizable competitive social game with identity, rivalry, and community.

The game should include systems that make players want to return.

8.1 Points and Ranking

Players earn points from:

- round wins,

- match wins,
- tournament placements,
- win streaks,
- defeating higher-ranked players,
- surviving multiple rounds,
- winning after a comeback,
- winning with rarely used choices.

Example:

Zain defeated a higher-ranked player.
+40 points

8.2 Leaderboards

Leaderboards create competition and social pressure.

Possible leaderboard types:

- daily leaderboard,
- weekly leaderboard,
- monthly leaderboard,
- server leaderboard,
- global leaderboard,
- friends-only leaderboard,
- country leaderboard,
- university leaderboard.

This gives players a reason to return and protect their rank.

8.3 Titles and Identity

Players can earn funny or legendary titles based on their behavior.

Examples:

- Rock Addict,
- Scissors Demon,
- Paper Merchant,
- Comeback King,

- First Round Victim,
- Final Boss,
- Chaos Player,
- The Unpredictable One,
- Restaurant Bill Survivor,
- Pizza Debt Champion.

These titles make players feel like they have an identity inside the game.

8.4 Streaks

Streaks are one of the strongest return mechanics.

Examples:

- win streak,
- loss streak,
- daily play streak,
- tournament win streak,
- revenge streak against the same opponent.

MemeForge can react differently depending on the streak.

Example:

Zain has won 5 matches in a row.

Someone stop him before this becomes his personality.

8.5 Rivalries

The game should track repeated matches between the same players.

Example:

Zain vs Adam

Matches played: 17

Zain wins: 9

Adam wins: 8

Current rivalry status: Dangerous

This creates personal stories between players.

8.6 Events and Seasons

The game can later include limited-time events.

Examples:

- weekend tournament,
- Ramadan edition,
- university server championship,
- 42 Vienna tournament,
- Discord server war,
- country vs country tournament,
- restaurant bill championship,
- pizza night arena.

Seasons allow rankings to reset, giving new players a chance to compete.

8.7 Brand Expansion

If the game grows, the identity system can connect to merchandise.

Examples:

- T-shirts with player titles,
- hoodies for tournament winners,
- stickers with MemeForge reactions,
- server champion badges,
- limited seasonal designs.

This turns the game from a simple app into a small culture.

9. MemeForge — AI Reaction Engine

MemeForge is the AI-powered reaction engine of Rock Paper Scissors Universe.

It creates personalized memes, GIFs, images, short videos, audio reactions, captions, roasts, and shareable content from game results.

In simple words:

Game Engine = decides who wins.

MemeForge = makes the win/loss funny and shareable.

MemeForge is not only a meme generator.

It is a game-moment engine.

It turns results into memories.

9.1 Meaning of the Name

Meme = funny/shareable internet reaction.

Forge = a workshop where something is created.

So MemeForge means:

A workshop that creates memes from game moments.

This name is stronger than MemeGen because MemeGen sounds generic.

MemeForge sounds like a creative engine inside the game universe.

9.2 What MemeForge Creates

MemeForge can generate:

- funny captions,
- loser roasts,
- winner celebrations,
- GIFs,
- images,
- short videos,

- audio reactions,
- tournament posters,
- champion cards,
- shareable social media posts,
- merch-ready designs.

Example loser reaction:

Zain picked Scissors 5 times in a row.
Paper was waiting patiently.

Example winner reaction:

Zain has defeated 7 players.
The restaurant bill fears him.

9.3 Two Emotional Registers

MemeForge should have two main emotional registers.

9.3.1 Loser Register — The Roast

The loser reaction should be funny, teasing, and unexpected.

It should not be too cruel, but it should feel like the game is joking with the player.

Examples:

Zain could not beat a rock.

You picked Scissors again.
At this point, it is not strategy. It is a lifestyle.

Paper appeared.
Zain disappeared.

9.3.2 Winner Register — The Silly Hero

The winner reaction should be over-the-top and dramatic.

It should treat the player like a legendary hero, but in a ridiculous way.

Examples:

Zain has defeated 6 opponents.

None of them saw it coming.

Not even Zain.

The arena bows.

The bill trembles.

The champion rises.

Adam stands alone.

Master of Rock.

Destroyer of restaurant debts.

9.4 Offline Mode — Pre-Installed Meme Pack

When there is no internet connection, MemeForge uses a pre-installed meme pack.

This pack is bundled with the app.

It includes classic meme templates stored locally.

Offline mode cannot generate fully new AI images, but it can still personalize templates using:

- username,
- score,
- rank,
- match result,
- choice pattern,
- tournament moment.

The offline pack should include:

- 30–50 meme templates,
- win templates,
- loss templates,
- draw templates,
- shock templates,
- domination templates,

- comeback templates,
 - elimination templates.
-

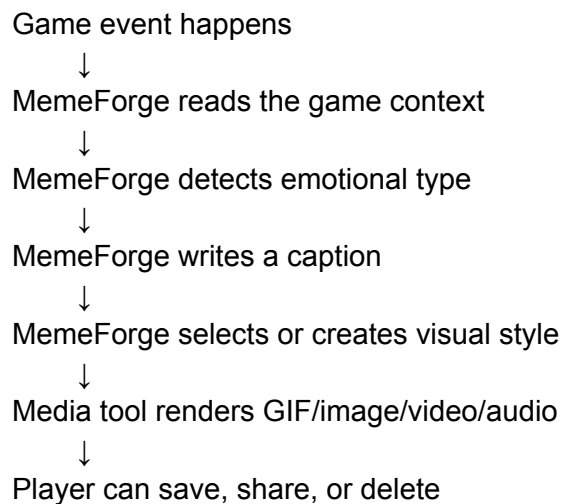
9.5 Online Mode — AI Generation

When internet is available, MemeForge uses the full AI pipeline.

It can generate or customize:

- meme captions,
- image prompts,
- GIF concepts,
- short video concepts,
- audio reactions,
- personalized roasts,
- winner posters,
- tournament summaries.

Possible AI pipeline:



9.6 Personalization Data

MemeForge can use:

Data

Usage

Username	Inserted into captions
Avatar	Overlaid onto meme or GIF
Rank	Used for status jokes
Win/loss streak	Used for dramatic reactions
Choice pattern	Used for targeted jokes
Match context	Changes the joke
Tournament round	Makes final rounds feel bigger
Rivalry history	Creates personal jokes
Server name	Makes sharing more social

Example:

Zain has picked Rock 8 times today.

At this point, he is not playing Rock Paper Scissors.
He is applying for a geology degree.

10. Meme Ownership, Sharing, and Merch System

MemeForge should not generate a meme and forget it.

Generated memes, GIFs, images, and short videos can become part of the player's identity.

10.1 Saving Generated Memes

After a round, match, or tournament, the game can ask the player if they want to save the generated meme.

Possible options:

- save to profile,
- download,
- share to Discord,
- share to Telegram,
- share to Instagram,
- share to TikTok,
- use as profile badge,
- add to favorite memes,
- use later for merch.

This turns memes into collectibles.

10.2 Winner Meme Choice

The winner should sometimes be allowed to choose their favorite generated meme from the tournament.

Example:

Tournament finished.

Choose your champion meme:

1. Victory GIF
2. Final Boss Poster
3. Restaurant Bill Destroyer Meme
4. Legendary Rock Moment

The selected meme can be saved as the player's Champion Moment.

10.3 Social Sharing

Every generated meme should be easy to share.

A meme can include:

- player name,
- tournament name,
- rank,

- win streak,
- funny title,
- date,
- server name,
- QR code,
- invite link,
- game logo.

Example share text:

Zain won the 42 Vienna RPS Tournament.

Title unlocked:

The Paper Merchant

Challenge him if you dare.

This helps the game spread naturally because players share their funniest moments.

10.4 Meme Gallery

Each player can have a personal gallery.

The gallery stores:

- best win memes,
- worst loss memes,
- tournament champion memes,
- rivalry memes,
- comeback memes,
- rare event memes,
- seasonal memes.

This creates emotional memory.

Players are not only collecting points.

They are collecting funny moments.

10.5 Merch System

If the game grows, saved memes can become physical products.

Possible merch:

- T-shirts,
- hoodies,
- stickers,
- posters,
- mugs,
- tournament champion cards,
- limited edition event shirts.

A high-ranked player can choose one of their saved memes and print it on a shirt.

Example:

Front:

Zain — The Scissors Demon

Back:

Defeated 12 players.

Still lost to Paper once.

Another example:

Champion of the 42 Vienna Arena

Rock Paper Scissors Universe

Season 1

10.6 Rank-Based Merch Unlocks

Not every merch item should be available immediately.

Some can unlock based on rank or achievement.

Examples:

Bronze Rank:

Basic profile sticker

Silver Rank:

Custom meme poster

Gold Rank:
Champion T-shirt

Diamond Rank:
Limited hoodie

Tournament Winner:
One exclusive champion design

This gives players another reason to compete.

10.7 Important Rule for AI-Generated Media

The system should save only memes that the user chooses to keep.

Not every generated meme needs to be permanently stored.

Recommended flow:

AI generates meme
↓
Player sees result
↓
Player chooses Save / Share / Delete
↓
Saved meme goes to profile gallery
↓
Favorite meme can later be used for merch

This keeps storage cleaner and gives players control over their identity.

10.8 Long-Term Meme Vision

The meme system can become one of the most powerful parts of the project.

The game is not only about winning.

It is about creating moments that players want to show other people.

The goal is:

Every match should create a story.
Every tournament should create a legend.
Every legend should create a meme.
Every meme should be shareable.

11. Plugin Architecture — Reusable Engines

The project should be designed so that some parts can later be reused in other games.

The three main reusable systems are:

- MemeForge,
 - Tournament Engine,
 - Scoring and Taunts Plugin.
-

11.1 MemeForge Plugin

MemeForge should work as a plugin.

Any game can send it an event object.

MemeForge reads the event and creates a reaction.

Example event object:

event_type: win / loss / draw / elimination / comeback / upset

player:

username
avatar_url
rank
stats
play_pattern

game_context:

game_name
round_number
match_score
opponent
tournament_stage

MemeForge then generates:

- text reaction,
 - image,
 - GIF,
 - short video,
 - audio reaction,
 - or shareable post.
-

11.2 Scoring and Taunts Plugin

This plugin tracks player scores across sessions.

It can work with or without MemeForge.

It generates text-only reactions if media is not available.

Useful for:

- Discord live commentary,
- terminal mode,
- low-resource mode,
- offline mode.

Example:

Zain has lost 3 times in a row.
The comeback arc begins now.

11.3 Tournament Engine Plugin

The tournament engine should be reusable.

Any game can give it a list of players.

The tournament engine handles:

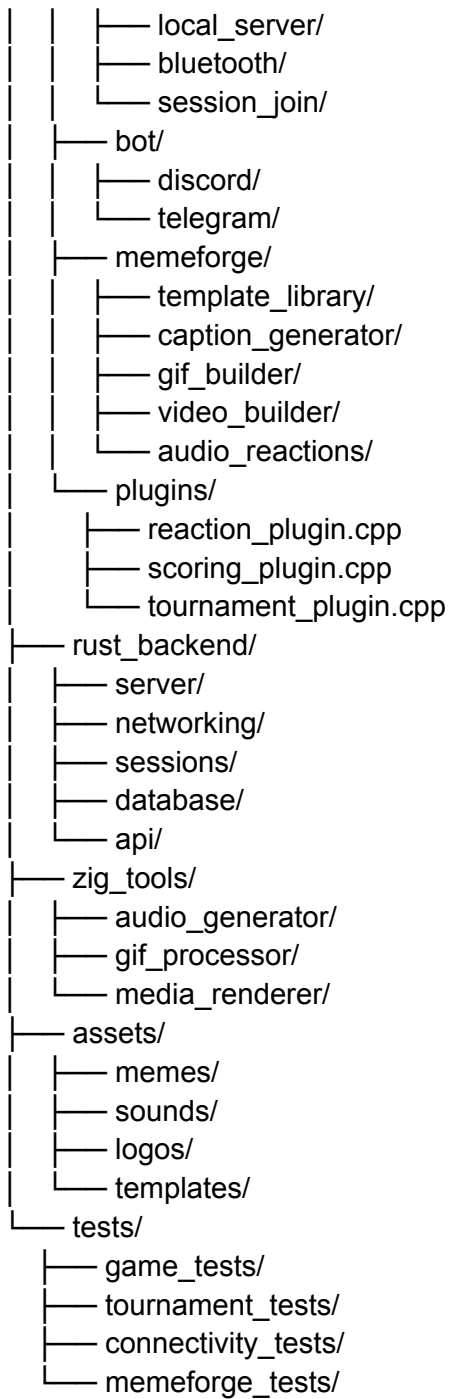
- shuffling,
- pairing,
- odd-player handling,
- brackets,
- round advancement,
- elimination,
- winner determination.

The game only needs to provide the match result.

12. Project Folder Structure

Recommended folder structure:

```
rps/
├── Makefile
├── README.md
├── docs/
│   └── project_vision.md
├── include/
│   ├── game/
│   ├── tournament/
│   ├── connectivity/
│   ├── plugins/
│   └── memeforge/
├── src/
│   ├── game/
│   │   ├── game_logic.cpp
│   │   ├── player.cpp
│   │   ├── computer.cpp
│   │   └── display.cpp
│   ├── ui/
│   │   ├── terminal_ui.cpp
│   │   └── ui_adapter.cpp
│   ├── tournament/
│   │   ├── tournament_manager.cpp
│   │   ├── bracket.cpp
│   │   └── player_registry.cpp
│   └── connectivity/
```



13. Technology Stack

Area

Technology

Core game engine	C++17
Build system	GNU Make
Networking/backend	Rust
Local networking	TCP / WebSocket / local server
Bluetooth	BLE / platform-specific libraries
Database	SQLite early, PostgreSQL later
Temporary sessions	Redis
Discord bot	DPP or Discord API adapter
Telegram bot	Telegram Bot API / tgbot-cpp
Meme captions	AI text model
Image generation	DALL-E / Stable Diffusion / other image API
GIF creation	giflib or custom media pipeline
Audio generation	Zig tools or external audio API
Video generation	AI video API or local renderer
Testing	Catch2 for C++, Rust tests for backend
Assets	Local meme pack, sound pack, templates

14. Development Roadmap

Phase 1 — C++ Core Game

Current phase.

Goals:

- split the original single-file game into modules,
- create clean header files,
- write a Makefile,
- keep the game working in terminal,

- make the code understandable and expandable.

Deliverable:

Working terminal Rock Paper Scissors game using multiple C++ files.

Phase 2 — C++ Tournament Engine

Goals:

- add tournament mode,
- add player registry,
- add random pairing,
- add odd-player handling,
- add elimination logic,
- add scoring,
- add simple terminal tournament display.

Deliverable:

Local terminal tournament with multiple players.

Phase 3 — Rust Networking Layer

Goals:

- create local multiplayer server,
- handle player connections,
- manage sessions,
- receive private choices,
- synchronize simultaneous reveal,
- send results back to players.

Deliverable:

Local multiplayer game over WiFi or hotspot.

Phase 4 — Rust Backend and Database

Goals:

- add persistent player profiles,
- store stats,
- store ranks,
- store tournament history,
- store rivalry history,
- connect backend with database,
- prepare online sessions.

Deliverable:

Backend service with player accounts, stats, and session history.

Phase 5 — Discord Bot and MemeForge

Goals:

- build Discord bot,
- support slash commands,
- support join by PIN,
- support tournament mode inside Discord,
- add MemeForge offline mode,
- add MemeForge online mode,
- generate captions and GIF reactions,
- allow players to save/share memes.

Deliverable:

Live Discord Rock Paper Scissors tournament bot with MemeForge reactions.

Phase 6 — Zig Media Tools

Goals:

- experiment with Zig for audio generation,
- build small media tools,
- process GIF frames,
- create sound effects,
- support faster local rendering tools.

Deliverable:

MemeForge media helper tools written in Zig.

Phase 7 — Telegram and Web Platform

Goals:

- add Telegram adapter,
- add invite link joining,
- add basic web dashboard,
- allow players to see rankings,
- allow players to view meme gallery.

Deliverable:

Same game works on Discord, Telegram, and web.

Phase 8 — Social Sharing and Merch

Goals:

- add player meme gallery,
- add favorite meme system,
- add shareable champion cards,
- add social media export,
- add merch-ready designs,
- allow high-ranked players to print favorite memes on shirts or hoodies.

Deliverable:

Players can save, share, and use generated game moments for merch.

15. Immediate Next Steps

For the current school/project version, focus only on Phase 1.

Do not start Rust, Zig, Discord, AI, or database yet.

The immediate goal is to build a clean C++ foundation.

15.1 Files to Create First

Create this structure:

```
rps/
├── Makefile
├── include/
│   ├── game_logic.h
│   ├── player.h
│   ├── computer.h
│   └── display.h
└── src/
    ├── main.cpp
    ├── game_logic.cpp
    ├── player.cpp
    ├── computer.cpp
    └── display.cpp
```

15.2 Module Responsibilities

game_logic.h / game_logic.cpp

Contains:

- enums,
- structs,
- round winner logic,
- final winner logic,
- main game flow.

Possible functions:

WhoWonTheRound
WhoWonTheGame
StartGame

player.h / player.cpp

Contains:

- reading player choice,
- validating player input,
- converting input to game choice.

Possible function:

ReadPlayerChoice

computer.h / computer.cpp

Contains:

- random number generation,
- computer choice generation.

Possible functions:

RandomNumber

GetComputerChoice

display.h / display.cpp

Contains:

- printing round results,
- printing final results,
- converting enum values to readable names.

Possible functions:

ChoiceName

WinnerName

PrintRoundResults

PrintGameResults

main.cpp

Should stay very small.

It should only start the program.

Example responsibility:

main calls StartGame.

15.3 Phase 1 Rule

The first phase should not add advanced features.

Do not add:

- Discord,
- Rust server,
- Zig tools,
- AI,
- database,
- online multiplayer,
- QR code,
- MemeForge.

Only build the clean C++ terminal game.

The rule:

Make the foundation clean before building the universe.

16. Final Vision Statement

Rock Paper Scissors Universe is not just a Rock Paper Scissors game.

It is a social decision arena.

It turns small everyday choices into funny competitive moments.

It starts as a simple C++ terminal project and grows into a cross-platform tournament system powered by:

- C++ for the game brain,
- Rust for networking and backend,
- Zig for creative media tools,
- MemeForge for AI reactions,
- Discord and Telegram for community play,
- leaderboards for competition,
- memes for identity,
- and merch for long-term brand expansion.

The final vision:

One engine.

Every platform.

Every room.

Every match creates a story.

Every story creates a meme.

Every meme can become a legend.

17. Project Slogan Options

Possible slogans:

Tiny decisions. Legendary battles.

When humans cannot decide, they must duel.

Every match creates a story.

Every story creates a meme.

Rock. Paper. Scissors. Glory.

The ancient ritual for modern problems.

Best current slogan:

Tiny decisions. Legendary battles.

18. Current Official Names

System	Name
Main game	Rock Paper Scissors Universe
AI reaction engine	MemeForge
Tournament system	Tournament Engine
Scoring system	Scoring and Taunts Plugin
Local/online backend	Rust Session Layer
Media/audio tools	Zig Media Tools

19. Final Short Summary

Rock Paper Scissors Universe begins as a C++ learning project, but the long-term idea is to build a funny, competitive, social, cross-platform game.

The core game decides the winner.

The tournament engine creates competition.

The Rust backend connects players safely.

The Zig tools help generate media.

MemeForge turns results into funny shareable moments.

The community system creates rankings, rivalries, titles, and identity.

The merch system turns saved memes into real-world products.

The project starts small, but the vision is big:

A simple hand game becomes a social universe.

20. Additional Game Vision Ideas

20.1 Machine Opponent — Dark Funny Story Mode

When a player plays against the machine, the experience should not feel like playing against a normal computer.

The machine should feel like a mysterious opponent.

Before or during the match, the game can tell the player a funny, dark, scary story to build tension.

The goal is not real horror. The goal is comedy mixed with dramatic fear.

Example:

You have chosen to face the machine.

Many players entered before you.

Some chose Rock.

Some chose Paper.

Some chose Scissors.

None of them were the same after the restaurant bill arrived.

The machine is watching.

The machine is calculating.

The machine already knows you are thinking about Scissors.

This makes player-vs-machine mode more memorable.

The machine can also speak differently depending on the player's behavior.

Example if the player keeps choosing Rock:

Again with Rock?

The machine sighs in binary.

Somewhere, a mountain is disappointed.

Example if the player loses:

The machine has defeated you.

Do not worry.

Many brave humans have fallen before.

Mostly because they were too confident with Paper.

Example if the player wins:

Impossible.

The machine has paused.

The machine has blinked.

The machine is pretending this was part of the plan.

The machine personality should be:

- funny,
 - dark,
 - dramatic,
 - sarcastic,
 - slightly scary,
 - never truly harmful,
 - always entertaining.
-

20.2 Reusable Decision Game Engine

Rock Paper Scissors Universe should later become more than one game.

The core decision mechanic should be reusable by other games.

This means the Rock Paper Scissors engine can become a plugin that other games can use when they need a quick decision.

Examples:

- who starts first,
- who attacks first,
- who gets a bonus,
- who gets punished,
- who chooses the next map,
- who receives a random challenge,
- who pays in a funny party game,
- who becomes the leader for the next round.

The idea:

Rock Paper Scissors becomes a universal decision engine.

Other games can plug it in like this:

Game needs a decision



RPS Decision Engine starts



Players choose Rock / Paper / Scissors



Winner is returned to the game



The original game continues

This makes the project more powerful because it is not only a standalone game.

It becomes a tool that can be used inside other games.

20.3 Infrastructure Vision — Built for Massive Scale

From the beginning, the project should be designed with future scale in mind.

The first version will be small.

It may only run in the terminal.

But the architecture should not block future growth.

The long-term infrastructure goal is to support:

- thousands of players,
- millions of players,
- global tournaments,
- live events,
- many servers,
- many sessions at the same time,
- many platforms at the same time.

The dream goal is:

A system that can one day handle hundreds of millions of players.

This does not mean Phase 1 must already support that.

It means the design should avoid bad decisions that would make scaling impossible later.

Important infrastructure principles:

- separate the game engine from networking,
- separate the backend from the frontend,
- separate session state from permanent storage,
- use Rust for safe networking and concurrency,
- use Redis or similar tools for fast temporary session state,
- use PostgreSQL or similar databases for permanent data,
- use load balancing later,
- use distributed servers later,
- keep the core game logic stateless when possible,
- make every session independent,
- design APIs cleanly from the beginning.

The rule:

Start small, but design like it may become global.

20.4 Global University Day / World Event

The game can later have a special global event day.

On this day, players from different places can join one huge tournament or event.

Possible event names:

World Decision Day

Global RPS Day

University Arena Day

The Great Decision Day

Rock Paper Scissors World Clash

Universities, schools, Discord servers, and online communities can join.

The event can work like this:

Players join from around the world



Local groups or servers compete



Winners move to regional rounds



Regional winners move to global rounds



One final winner is crowned

The final winner can receive:

- global champion title,
- special profile badge,
- exclusive MemeForge champion poster,
- special hoodie or T-shirt,
- social media share card,
- place in the Hall of Legends.

Example final announcement:

The world has decided.

After thousands of battles,
millions of choices,
and one suspiciously powerful Paper streak,

Zain has become the Global Decision Champion.

This event can make the game feel bigger than a normal app.

It becomes a community moment.

20.5 Multi-Language Player Experience

Players should be able to choose their system language.

The game should support different languages in the future.

Examples:

- English,
- German,
- Arabic,
- French,
- Romanian,
- Turkish,
- Spanish,
- and more.

If two players use different languages, each player can receive the game messages in their own language.

Example:

Player 1 language: English

Player 2 language: German

Player 1 sees:

You lost.

Paper has betrayed you.

Player 2 sees:

Du hast gewonnen.

Papier hat heute Geschichte geschrieben.

MemeForge can also generate memes in each player's language.

The system should store each player's language preference.

Possible language settings:

player_language

meme_language

voice_language

subtitle_language

This makes the game more global and more personal.

20.6 Cross-Language MemeForge

MemeForge should understand that different players may speak different languages.

If a meme is generated after a match, the system can create:

- one version for Player 1,
- one version for Player 2,
- or one shared bilingual version.

Example bilingual meme:

Zain thought Rock would save him.
Stein war heute leider arbeitslos.

For global tournaments, MemeForge can use:

- local jokes,
- language-specific humor,
- translated captions,
- subtitles,
- voice lines,
- different cultural references.

This makes the game more international and more shareable.

20.7 Rock, Paper, and Scissors as Flexible Symbols

Rock, Paper, and Scissors are the main symbols of the game.

But later, they can be reskinned with different themes, text, speech, and actions.

The logic stays the same:

Rock beats Scissors.
Scissors beats Paper.
Paper beats Rock.

But the presentation can change.

Examples:

Classic Mode

Rock
Paper
Scissors

Medieval Mode

Shield
Scroll
Sword

Food Mode

Pizza
Bill
Wallet

Robot Mode

Metal
Code
Laser

Ancient Arena Mode

Stone
Law Scroll
Blade

The player still plays the same game, but the feeling changes.

This allows the game to have:

- skins,
- seasonal events,
- funny modes,
- cultural versions,
- special tournament themes,
- unlockable packs.

The rule:

The logic stays simple.
The world around it can change forever.

20.8 Different Actions and Voice Lines

Each choice can have different animations, text, and voice lines.

Example Rock actions:

Rock has entered the arena.
Rock refuses to explain itself.
Rock crushes Scissors with ancient confidence.

Example Paper actions:

Paper floats in silently.
Paper signs the contract of victory.
Paper covers Rock like a legal document.

Example Scissors actions:

Scissors arrives dramatically.
Scissors cuts through Paper's dreams.
Scissors came sharp and emotionally unstable.

These actions can change based on:

- theme,
- player rank,
- tournament stage,
- win streak,
- language,
- event type,
- MemeForge style.

This gives the same simple game many different feelings.

20.9 Updated Long-Term Vision

The project is not only a Rock Paper Scissors game.

It is:

- a decision engine,
- a tournament system,
- a social arena,
- a meme generator,
- a global event platform,
- a multilingual game experience,
- and a future plugin that other games can use.

The long-term idea:

Any small decision can become a battle.

Any battle can become a story.

Any story can become a meme.

Any meme can become a memory.

Any memory can become part of the player's identity.